



Package dan Modularization

Materi ini membahas package, module, dan struktur folder pada Go agar backend lebih rapi, mudah dipelihara, dan dikembangkan.



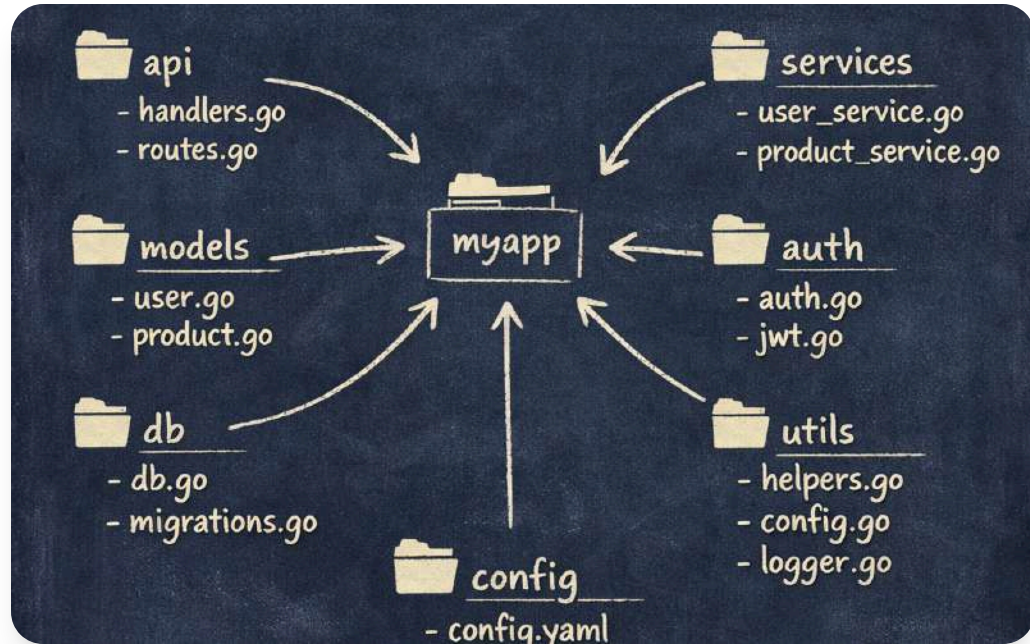
Juara Coding

June 2026



Tujuan Pembelajaran

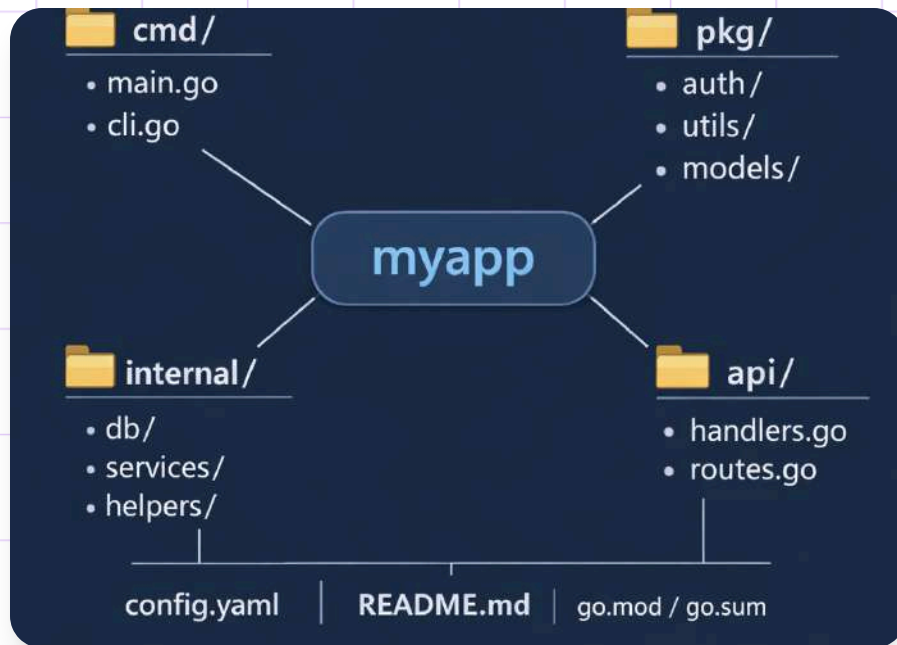
Peserta memahami package Go, beda package dan module, struktur project, impor package sendiri, dan kode backend rapi.



Package di Go

Package di Go mengelompokkan kode berdasarkan fungsi, menyatukan file terkait agar struktur backend lebih rapi, jelas, dan mudah dikelola tim.

Package main dan Package Lain



Dalam Go, **package main** menjadi titik awal eksekusi, sedangkan package lain menyimpan logika agar struktur program lebih jelas.

01
10

Package main

Menjadi titik awal program yang dapat dijalankan dan mengatur alur eksekusi utama.



Package Fungsional

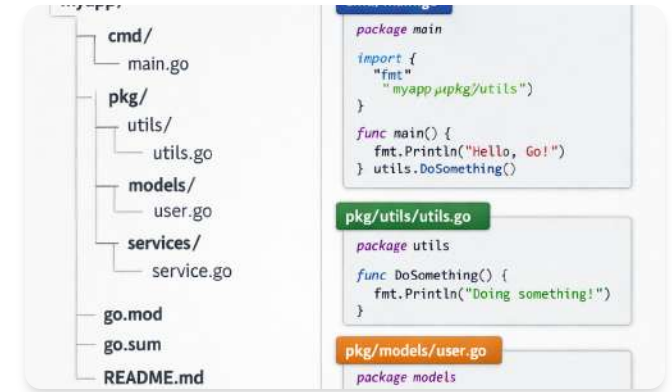
Digunakan untuk logika tertentu seperti auth, user, atau database agar kode lebih terorganisir.



Pemisahan Tanggung Jawab

Membuat program lebih jelas antara bagian eksekusi dan bagian fungsional.

Aturan Dasar Package



01 Nama Package

Nama package biasanya singkat, jelas, dan mewakili isi folder tempat file berada.

02 Konsistensi Folder

Semua file dalam satu folder umumnya menggunakan nama package yang sama untuk menjaga struktur tetap rapi.

03 Akses Identifier

Nama fungsi atau variabel berawalan huruf besar dapat diakses lintas package, sedangkan huruf kecil bersifat lokal.

Export dan Visibility

Unexported	Exported
• Name starts with a lowercase letter • Accessible only within the same package • Not visible to other packages	• Name starts with an uppercase letter • Accessible from other packages • Visible outside the package
 Internal Only	 Accessible Everywhere

Di Go, nama berawalan huruf besar bersifat *exported* untuk antar package, sedangkan huruf kecil hanya untuk package yang sama.

01
10

Huruf besar = exported

Fungsi seperti *CreateUser* bisa dipanggil dari package lain karena diawali huruf besar.

01
10

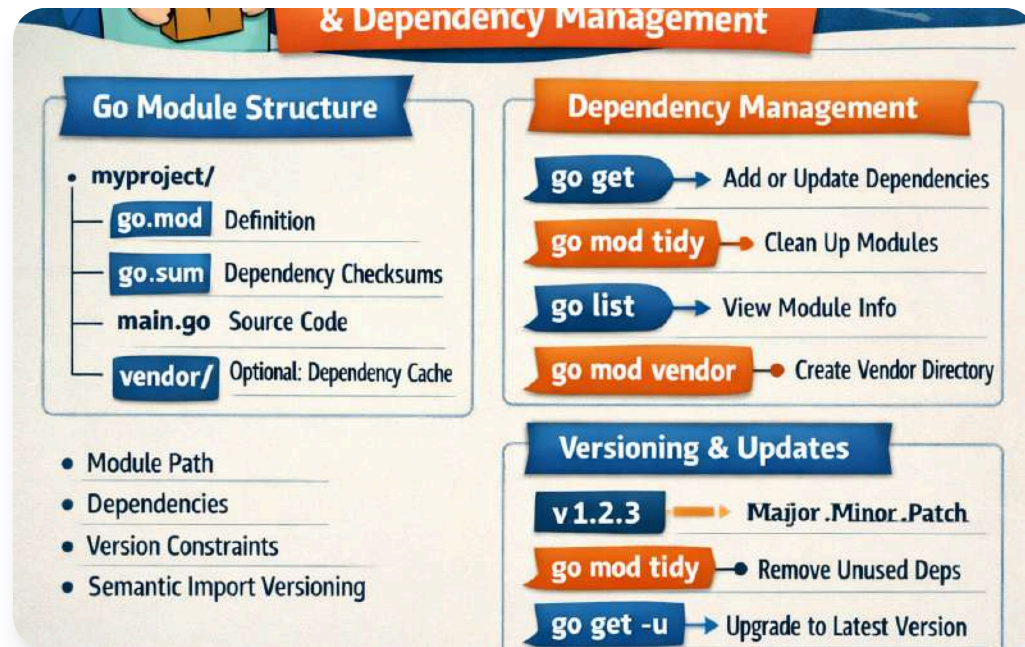
Huruf kecil = internal

Fungsi seperti *createUserHelper* hanya bisa digunakan di dalam package yang sama.



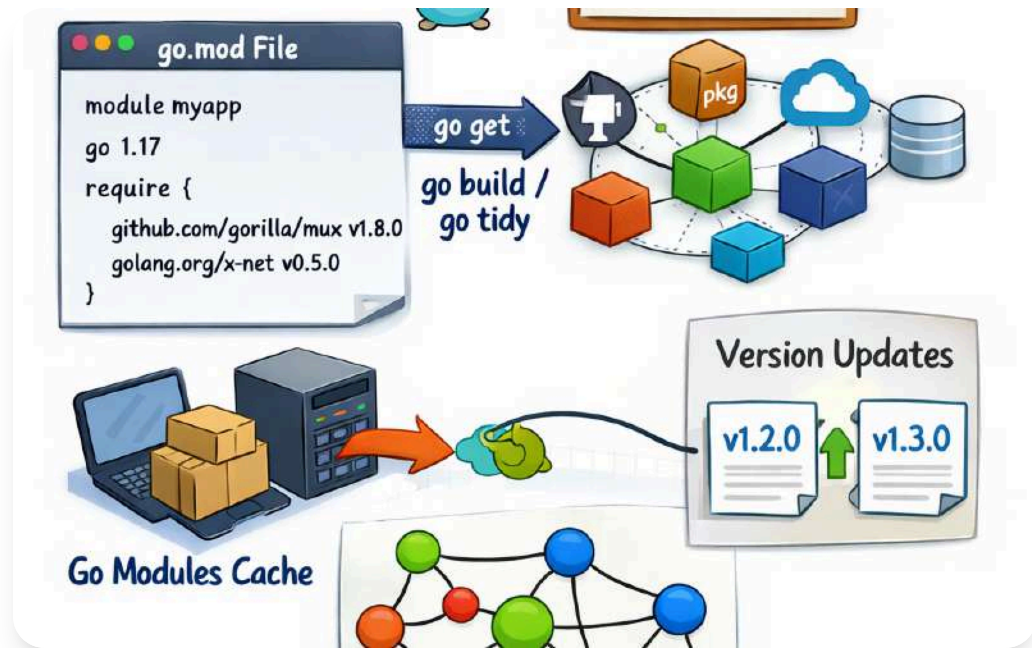
Menjaga batas akses

Aturan ini membantu modularisasi dan menjaga agar akses kode tetap terkontrol.



Apa Itu Module

Module adalah satuan project Go dengan file *go.mod* untuk identitas project dan pengelolaan dependency secara terstruktur.



Peran *go.mod* dalam Project

File *go.mod* menyimpan nama module dan dependency agar Go dapat mengenali import, versi library, dan konsistensi project.

Contoh Struktur Project

Folder/File	Peran
main.go	Entry point aplikasi
go.mod	Manajemen module
user	Package fitur user
product	Package fitur product

Struktur project dasar dimulai dari **main.go**, **go.mod**, dan folder fitur agar kode lebih rapi, terpisah, dan mudah dibaca.

```
import (  
    "fmt"  
    "mypackage"  
)  
  
func main() {  
    msg == mypackage.Hello("Alice")  
    fmt.Println(msg)  
}
```

Import Package Sendiri

Package user bisa dipanggil dari **main** jika module dan folder sudah benar.

```
package mymath

// Add adds two integers and returns the sum.
func Add(a, b int) int {
    return a + b
}

// main.go
package main
import (
    "fmt"
    "mypackage/mymath"
}

func main() {
```

Package user Export

Fungsi *GetUserName* diawali huruf besar sehingga dapat dipanggil dari package lain, menunjukkan modularisasi logika secara sederhana.

Manfaat Modularization



Mudah Dirawat

Struktur modular membuat kode backend lebih rapi dan lebih mudah dipelihara seiring perkembangan aplikasi.



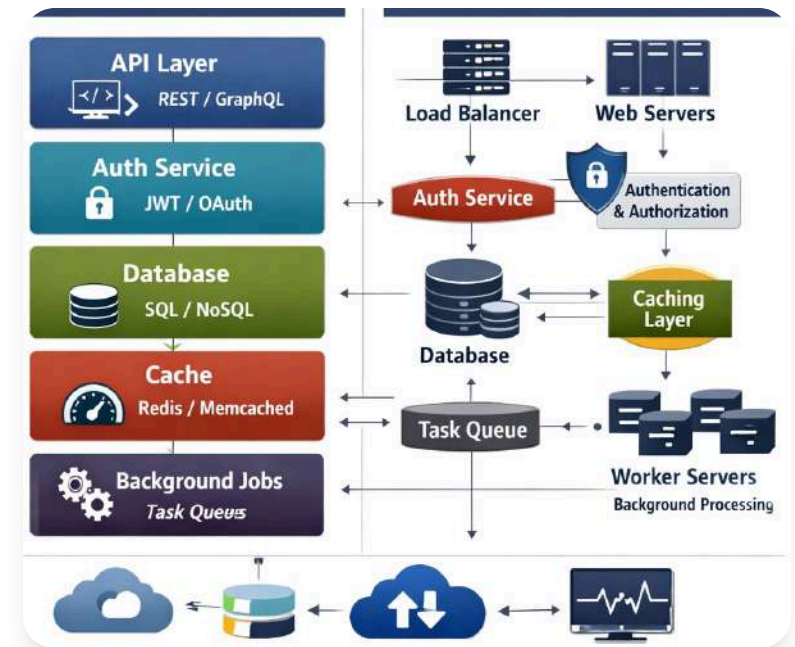
Pengembangan Lebih Aman

Perubahan fitur bisa dilakukan pada modul terkait tanpa mengganggu keseluruhan project.

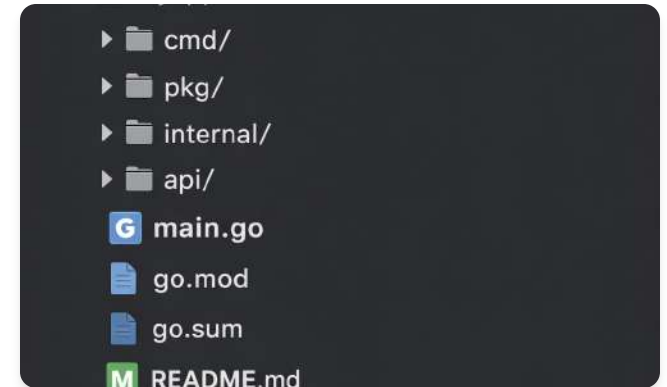


Lebih Mudah Diuji

Logika seperti user, auth, dan product dapat dipisahkan agar pengujian tiap bagian lebih fokus.



Kesalahan Umum yang Perlu Dihindari



01 Nama package tidak konsisten

Gunakan penamaan package yang seragam agar kode mudah dipahami dan tidak membingungkan tim.

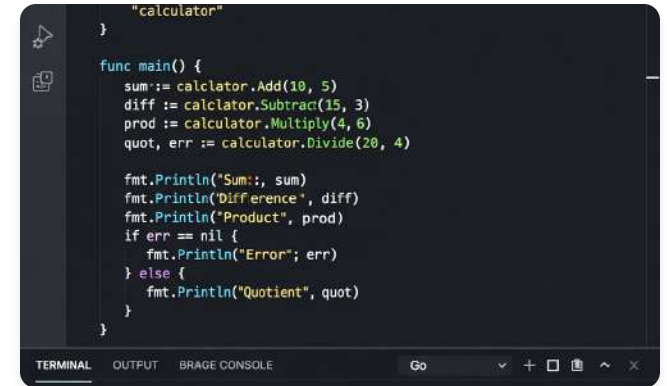
02 Struktur dan import tidak selaras

Pastikan import path sesuai dengan go.mod, dan hindari folder yang tidak mencerminkan fungsi kode.

03 Package main terlalu berat

Jangan menaruh terlalu banyak logika di package main atau memecah package terlalu kecil tanpa kebutuhan jelas.

Latihan Mini Project 1



```
package calculator

func main() {
    sum := calculator.Add(10, 5)
    diff := calculator.Subtract(15, 3)
    prod := calculator.Multiply(4, 6)
    quot, err := calculator.Divide(20, 4)

    fmt.Println("Sum:", sum)
    fmt.Println("Difference", diff)
    fmt.Println("Product", prod)
    if err == nil {
        fmt.Println("Error", err)
    } else {
        fmt.Println("Quotient", quot)
    }
}
```

01 Buat module project

Inisialisasi project Go dengan nama module *latihan/pertemuan6* sebagai dasar struktur project.

03 Panggil dari main.go

Import package internal lalu panggil semua fungsi dan tampilkan hasilnya untuk latihan penggunaan package.

02 Buat package calculator

Tambahkan fungsi *Add*, *Subtract*, *Multiply*, dan *Divide* sebagai fungsi export di package *calculator*.

Latihan Mini Project 2

// Book represents the schema for the book data.

```
type Book struct {
```

```
    ID      int    "json:"id""
```

```
    Title   string "json:"title"
```

```
    Author  string "json:"author"
```

```
    PublishedYear int "json:"published_year"
```

```
    ISBN    string "json:"isbn"
```

```
    Genre   string "json:"genre"
```

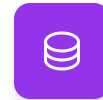
```
}
```

Buat backend dasar data buku dengan package book, penyimpanan slice sederhana, serta fitur tambah, daftar, dan cari berdasarkan ID.



Package book

Susun fungsi AddBook, ListBooks, dan FindBookByID dalam package terpisah.



Penyimpanan Sederhana

Simpan data buku di slice tanpa database untuk fokus pada logika dasar.



Integrasi main.go

Tampilkan daftar buku dan hasil pencarian ID untuk latihan modularization dasar.